

Global Semiconductor Industry

Exploratory Data Analysis (2010–2026)

Complete Code Guide

A block-by-block walkthrough of the Python code behind the notebook — what every line does, why it is written that way, and the data-visualisation principles it follows.

Notebook: `global-semiconductor-eda.ipynb` • Library stack: `pandas`, `numpy`, `matplotlib`

1. What this notebook does

The notebook is an **exploratory data analysis (EDA)** of five linked CSV files that together describe the global semiconductor industry from 2010 to 2026. The goal is to turn raw tables into a visual story in five acts:

- **Industry scale & leaders** — total revenue, split by business model, and the top companies.
- **The AI-chip boom** — accelerator revenue by vendor, market concentration, and compute growth.
- **Price dynamics** — memory price cycles vs. data-centre GPU prices.
- **The fab-capacity race** — wafer capacity by country and the leading-edge share.
- **Geopolitics** — export-control actions over time and by imposing country.

Every chart is produced with **matplotlib**; the data wrangling is done with **pandas**. The code is organised as one *setup* cell followed by one code cell per chart. The rest of this guide explains each of those cells in order.

The five datasets

File	Rows	What it holds
chip_companies_financials.csv	617	Revenue, margins, R&D, capex by company/segment/country
fab_capacity.csv	313	Monthly wafer capacity by company, country, process node
chip_prices.csv	405	Monthly prices: DRAM, NAND, HBM3, NVIDIA H100/B200
ai_chip_market.csv	120	AI-chip shipments, ASP, revenue, TFLOPS, TDP
export_controls.csv	34	Export-control actions with a 1-10 severity score

2. The setup cell — imports, style system, data loading

This first code cell runs once and prepares everything the charts need: libraries, a small reusable colour/style system, a portable data loader, and the five DataFrames.

2.1 Imports

```
import os, glob, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
from matplotlib.lines import Line2D
warnings.filterwarnings("ignore")
```

- **os, glob** — build file paths and *discover* where the data lives (used by the loader).
- **numpy (np)** — fast numeric arrays; used for the stacked-bar baseline accumulator.
- **pandas (pd)** — the workhorse: reads CSVs and does all grouping/reshaping.
- **matplotlib.pyplot (plt)** — draws every figure.
- **matplotlib.ticker (mticker)** — formats axis numbers (e.g. adds the '\$' and 'B').
- **warnings.filterwarnings("ignore")** — hides harmless library warnings so the output stays clean.

2.2 A tiny design system (colours)

Rather than pick colours ad-hoc per chart, the notebook defines them *once*. This keeps every figure visually consistent and colourblind-safe.

```
INK      = "#0b0b0b"  # primary text
INK_SOFT = "#52514e"  # secondary text
MUTED    = "#898781"  # axis / tick labels
GRID     = "#e1e0d9"  # hairline gridlines
SURFACE  = "#fcfcfb"  # chart background
CAT = ["#2a78d6", "#1baf7a", "#eda100", "#008300",
        "#4a3aa7", "#e34948", "#e87ba4", "#eb6834"]
BLUE, AQUA, YELLOW, GREEN, VIOLET, RED, MAGENTA, ORANGE = CAT
BLUE_RAMP = ["#cde2fb", "#9ec5f4", "#6da7ec", "#3987e5", "#2a78d6", "#1c5cab", "#104281"]
```

- **INK / INK_SOFT / MUTED / GRID / SURFACE** — the neutral 'ink & paper' tones for text, axes and background.
- **CAT** — eight categorical hues in a *fixed order*. The order is chosen so neighbouring colours stay distinguishable for colour-blind readers.
- The unpacking line gives each hue a readable name (BLUE, AQUA, ...) so the plotting code reads clearly.
- **BLUE_RAMP** — a single-hue light→dark ramp used when a colour must encode *magnitude* rather than identity.

2.3 Global chart defaults

```
plt.rcParams.update({
    "figure.facecolor": SURFACE, "axes.facecolor": SURFACE,
    "font.family": ["Segoe UI", "DejaVu Sans", "sans-serif"], "font.size": 11,
    "text.color": INK, "axes.labelcolor": INK_SOFT, "axes.edgecolor": "#c3c2b7",
    "xtick.color": MUTED, "ytick.color": MUTED, "axes.linewidth": 0.8,
    "figure.dpi": 110, "axes.titlesize": 14, "axes.titleweight": "bold",
})
```

rcParams is matplotlib's global settings dictionary. Setting it once means every chart automatically shares the same background, font, colours, and resolution — no need to repeat styling in each cell.

2.4 The style() helper

Every chart calls this one function to apply consistent 'chrome' (the frame, grid, title and labels), so the styling logic lives in a single place.

```
def style(ax, title=None, sub=None, ylab=None, xlab=None):
    for s in ("top", "right"):
        ax.spines[s].set_visible(False)      # remove box top/right
    for s in ("left", "bottom"):
        ax.spines[s].set_color("#c3c2b7")    # soften remaining axes
    ax.grid(axis="y", color=GRID, linewidth=0.8, zorder=0)
    ax.set_axisbelow(True)                  # grid sits BEHIND the data
    if title:
        ax.set_title(title, loc="left", color=INK, pad=14 if sub else 8)
    if sub:
        ax.text(0, 1.015, sub, transform=ax.transAxes, ha="left",
               va="bottom", color=MUTED, fontsize=10.5)
    if ylab: ax.set_ylabel(ylab, color=INK_SOFT)
    if xlab: ax.set_xlabel(xlab, color=INK_SOFT)
    ax.tick_params(length=0)                # hide tick marks
    return ax
```

- Removes the top and right **spines** (the box lines) for a lighter, modern look.
- Draws a faint horizontal **grid** and pushes it *behind* the data with `set_axisbelow(True)`.
- Left-aligns the **title** and adds an optional grey **subtitle** just above the plot using axes-fraction coordinates (`transform=ax.transAxes`).
- Optionally sets the y/x axis labels and hides the little tick marks. Returns the axis so calls can be chained.

2.5 Portable data loading

The same notebook must run both on your machine and on Kaggle, where the data appears under `/kaggle/input/...`. This helper probes several likely locations and returns the first that contains a known file.

```
def find_data_dir():
    candidates = glob.glob("/kaggle/input/*/") + [
        "/kaggle/input/global-semiconductor-industry-2010-2026", ".",
    ]
    for c in candidates:
        if os.path.exists(os.path.join(c, "chip_companies_financials.csv")):
            return c
    return "."
```

```
DATA = find_data_dir()
fin = pd.read_csv(os.path.join(DATA, "chip_companies_financials.csv"))
ai = pd.read_csv(os.path.join(DATA, "ai_chip_market.csv"))
prices = pd.read_csv(os.path.join(DATA, "chip_prices.csv"))
fab = pd.read_csv(os.path.join(DATA, "fab_capacity.csv"))
ctrl = pd.read_csv(os.path.join(DATA, "export_controls.csv"))
```

- **glob.glob("/kaggle/input/*/")** lists every mounted dataset folder on Kaggle.
- The loop returns the first folder that actually contains the financials CSV; locally that resolves to `"."` (the current folder).
- **pd.read_csv** loads each file into a DataFrame — a table you can group, filter and reshape. The five variables (`fin`, `ai`, `prices`, `fab`, `ctrl`) are reused by every later cell.

3. Section 1 — industry scale & leaders

3.1 Total revenue over time (area + line)

```
by_year = fin.groupby("year")["revenue_usd_bn"].sum()

fig, ax = plt.subplots(figsize=(10, 4.6))
ax.fill_between(by_year.index, by_year.values, color=BLUE, alpha=0.12) # soft area
ax.plot(by_year.index, by_year.values, color=BLUE, lw=2.4)             # the line
ax.scatter([by_year.index[-1]], [by_year.values[-1]], color=BLUE, s=36) # end dot
ax.yaxis.set_major_formatter(
    mtticker.FuncFormatter(lambda v, _: f"${v:,.0f}B"))                # $1,151B
style(ax, "Total tracked semiconductor revenue", "2010-2026", ylab="Revenue")

cagr = (by_year.values[-1] / by_year.values[0]) ** (1/(len(by_year)-1)) - 1
```

- **groupby("year").sum()** collapses 617 company-rows into one revenue total per year.
- **fill_between** paints a translucent area under the line; **plot** draws the line; **scatter** marks the final point. Together they give a clean 'area chart with emphasis'.
- **FuncFormatter** with a lambda rewrites raw axis numbers as currency (the `:`, `.0f` adds thousands separators, no decimals).
- **CAGR** = compound annual growth rate: the constant yearly rate that turns the first value into the last over N-1 steps.

3.2 Revenue by business model (stacked area)

```
def family(seg):
    if seg == "foundry":           return "Foundry"
    if seg == "eda_software":      return "EDA tools"
    if seg.startswith("fabless"):  return "Fabless"
    if seg.startswith("idm"):      return "IDM"
    if seg.startswith("equipment"): return "Equipment"
    return "Other"
fin["family"] = fin["segment"].map(family)

fam_year = (fin.groupby(["year", "family"])["revenue_usd_bn"].sum()
            .unstack("family").fillna(0))
ax.stackplot(fam_year.index, [fam_year[c].values for c in order],
             labels=order, colors=[colors[c] for c in order])
```

- The raw data has 20 fine-grained segments. **family()** collapses them into five readable groups; **.map()** applies it to every row.
- **groupby([...]).sum().unstack("family")** pivots the data into a year × family grid; **fillna(0)** replaces missing combinations with zero.
- **stackplot** stacks the five families so the top edge equals total revenue and each band shows one group's contribution. Colours come from the fixed categorical palette.

3.3 Top 15 companies (horizontal bars)

```
latest = fin[fin.year == fin.year.max()].nlargest(15, "revenue_usd_bn")[:-1]
bars = ax.barh(latest["company_name"], latest["revenue_usd_bn"], color=BLUE)
bars[-1].set_color(VIOLET) # highlight the leader
for b, v in zip(bars, latest["revenue_usd_bn"]):
    ax.text(b.get_width() + 1.5, b.get_y() + b.get_height()/2,
            f"${v:,.0f}B", va="center") # value label at bar end
```

- **fin.year == fin.year.max()** keeps only the latest year; **nlargest(15, ...)** selects the 15 biggest companies.
- **[:-1]** reverses the order so the largest bar ends up on top in a horizontal chart.

- Because bars encode a *ranking* (magnitude, not identity) they use one hue; the single leader is re-coloured to draw the eye. The loop writes each value at the end of its bar.

3.4 R&D vs capex (two lines, one axis)

```
inv = fin.groupby("year")[["rd_spend_usd_bn", "capex_usd_bn"]].sum()
ax.plot(inv.index, inv["capex_usd_bn"], color=AQUA, marker="o", label="Capex")
ax.plot(inv.index, inv["rd_spend_usd_bn"], color=BLUE, marker="o", label="R&D spend")
ax.legend(loc="upper left", frameon=False)
```

- Both measures are summed per year and drawn as two lines on the **same y-axis**. They share the same unit (\$B), so a second axis is deliberately avoided — dual-axis charts distort comparisons.
- Distinct palette colours plus a **legend** identify the two series without relying on colour alone.

4. Section 2 — the AI-chip boom

4.1 Revenue by vendor (top 5 + Other)

```
rev = ai.groupby(["year", "vendor"])["estimated_revenue_usd_m"].sum()\
      .unstack("vendor").fillna(0)
totals = rev.sum().sort_values(ascending=False)
top = list(totals.head(5).index)          # 5 biggest vendors overall
rev_plot = rev[top].copy()
rev_plot["Other"] = rev.drop(columns=top).sum(axis=1) # rest -> 'Other'
pal = {name: CAT[i] for i, name in enumerate(top)}    # fixed entity->colour
pal["Other"] = MUTED
ax.stackplot(rev_plot.index, [rev_plot[s].values/1000 for s in series], ...)
```

- There are 11 vendors — too many for a readable stack. The code keeps the **top 5 by total revenue** and merges the rest into a grey **‘Other’** band.
- **pal** maps each vendor to a fixed palette slot, so a vendor keeps the *same colour* in every chart (colour follows the entity, never its rank).
- Dividing by 1000 converts \$millions to \$billions for the axis.

4.2 Market concentration (leader’s share)

```
share = rev.div(rev.sum(axis=1), axis=0) * 100 # each vendor's % of the year
leader = totals.index[0]                       # biggest vendor overall
ax.plot(share.index, share[leader].values, color=BLUE, marker="o")
ax.axhline(50, color=MUTED, ls=(0, (4, 4)))    # 50% reference line
ax.set_ylim(0, 100)
```

- **rev.div(rev.sum(axis=1), axis=0)** divides every cell by its row total, converting revenue into each vendor’s yearly market share.
- Only the leader’s share is plotted; a dashed **axhline** at 50% gives a ‘majority of the market’ reference. Fixing y to 0-100 keeps the scale honest.

4.3 Frontier compute (log scale)

```
ai["launch_year"] = pd.to_datetime(ai["launch_date"]).dt.year
frontier = ai.groupby("launch_year")["fp16_tflops"].max()
ax.plot(frontier.index, frontier.values, color=VIOLET, marker="o")
ax.set_yscale("log")
```

- **pd.to_datetime(...).dt.year** extracts the launch year from a date string.
- **groupby(...).max()** takes the single most powerful chip launched each year (the ‘frontier’).
- **set_yscale("log")** uses a logarithmic axis so exponential growth (which spans several orders of magnitude) shows as a roughly straight line.

5. Section 3 — price dynamics

Product prices span five orders of magnitude (a \$2 DRAM chip vs. a \$30,000 GPU). Two techniques let us compare them fairly: **indexing to a common base**, and a **log scale**.

5.1 Indexed price trends (base = 100)

```
prices["date"] = pd.to_datetime(prices["year_month"] + "-01")
for prod, g in prices.groupby("product"):
    g = g.sort_values("date")
    idx = g["price"].values / g["price"].values[0] * 100 # first month = 100
    ax.plot(g["date"], idx, color=pcolor.get(name, MUTED), label=name)
    ax.annotate(name, (g["date"].values[-1], idx[-1]), ...) # label at line end
```

- Each product's price is divided by its *first* value and $\times 100$, so every series starts at 100. Now the lines show **relative change**, letting cheap memory and pricey GPUs share one axis.
- The loop draws one line per product and writes the product name directly at the end of its line (**direct labelling** is easier to read than a legend for line charts).

5.2 Latest price on a log scale

```
latest_px = (prices.sort_values("date").groupby("product").tail(1)
              .sort_values("price"))
bars = ax.barh(latest_px["name"], latest_px["price"])
ax.set_xscale("log")
ax.text(..., f"${v:,.0f}" if v >= 100 else f"${v:,.2f}") # smart formatting
```

- **groupby("product").tail(1)** keeps each product's most recent price after sorting by date.
- A **log x-axis** compresses the huge range so a \$2 chip and a \$30,000 GPU both fit on one readable chart.
- The conditional f-string shows cents for cheap items and whole dollars for expensive ones.

6. Section 4 — the fab-capacity race

6.1 Capacity by country (stacked area)

```
iso = {"TWN":"Taiwan", "KOR":"South Korea", "USA":"United States", ...}
fab["country"] = fab["country_iso3"].map(iso).fillna(fab["country_iso3"])
cap = fab.groupby(["year", "country"])["monthly_wafer_capacity"].sum()\
      .unstack("country").fillna(0)
topc = list(cap.sum().sort_values(ascending=False).head(5).index)
capp = cap[topc].copy(); capp["Other"] = cap.drop(columns=topc).sum(axis=1)
```

- **iso** maps 3-letter country codes to full names; **.map(...).fillna(...)** keeps the code if a name is missing.
- Same 'top 5 + Other' pattern as the vendor chart, so the stack stays legible.
- Values are later divided by 1,000,000 to show *millions* of wafers per month.

6.2 Leading-edge capacity and its share

```
leading = fab[(fab["process_node_nm"] <= 7) & (fab["year"] == fab["year"].max())]
le = leading.groupby("country")["monthly_wafer_capacity"].sum().sort_values()

fab["is_leading"] = fab["process_node_nm"] <= 7 # boolean flag
node_year = fab.groupby(["year", "is_leading"])["monthly_wafer_capacity"].sum()\
            .unstack().fillna(0)
share_le = node_year[True] / node_year.sum(axis=1) * 100
```

- A process node ≤ 7 nm counts as **leading-edge** (the most advanced chips). The first block ranks countries by leading-edge capacity in the latest year.
- **is_leading** is a True/False column; grouping on it splits capacity into leading vs. mature, and the ratio gives the leading-edge **share** over time.

7. Section 5 — geopolitics

7.1 Export-control timeline (stem plot)

```
ctrl["date"] = pd.to_datetime(ctrl["date"])
imp_color = {"USA": BLUE, "NLD": ORANGE, "CHN": RED}
for country, g in ctrl.groupby("imposing_country"):
    ax.vlines(g["date"], 0, g["severity_score"], color=imp_color[country])
    ax.scatter(g["date"], g["severity_score"], color=imp_color[country],
               label=imp_name[country])
```

- Each action is drawn as a **stem**: **vlines** makes the vertical stalk from 0 up to the severity score, and **scatter** puts a dot on top.
- Grouping by imposing country and colouring by a fixed map (USA / Netherlands / China) shows at a glance who is driving the controls.

7.2 Actions per year (manual stacked bars)

```
cnt = ctrl.groupby(["year", "imposing_country"]).size()\
        .unstack("imposing_country").fillna(0)
bottom = np.zeros(len(cnt))
for country in cnt.columns:
    ax.bar(cnt.index, cnt[country], bottom=bottom, color=imp_color[country])
    bottom += cnt[country].values          # raise the base for the next layer
```

- **.size()** counts rows per (year, country) pair; unstacking makes a year × country grid of counts.
- Stacked bars are built by hand: each country is drawn starting at **bottom**, then **bottom** is raised by that country's counts so the next layer sits on top.

8. The design principles behind every chart

The styling is not decoration — it follows a small set of data-visualisation rules that make the charts accurate and accessible:

- **Colour by job.** Identity → fixed categorical hues; magnitude → one-hue ramp or a single colour. Colours are never cycled by rank.
- **One axis, never two y-scales.** Series with different units are compared by *indexing to a common base* or shown as separate charts (see the price section).
- **Colour follows the entity.** NVIDIA is the same blue in every chart; a filter that changes the number of series never repaints the survivors.
- **Log scale for wide ranges.** Used for frontier compute and unit prices, which span several orders of magnitude.
- **Recessive chrome.** Thin, soft axes; a faint grid behind the data; no chart-junk. The data is the loudest thing on the page.
- **Direct labelling.** Line ends and bar ends are labelled in place, reducing reliance on legends and colour alone.
- **Top-N + 'Other'.** When a category has too many values, keep the largest few and pool the rest, so stacks stay readable.

9. Running and publishing the notebook

9.1 Run it locally

```
python -m nbconvert --to notebook --execute --inplace \
    global-semiconductor-eda.ipynb
```

This executes every cell and saves the rendered charts back into the .ipynb file.

9.2 Publish to Kaggle

```
python -m kaggle kernels push -p \
    "C:\Users\vy355\OneDrive\Desktop\archive\kernel"
```

The `kernel` folder holds the notebook plus a `kernel-metadata.json` file that tells Kaggle the title, that it is public, and that it draws on the source dataset. Kaggle then re-runs the notebook on its own servers and publishes it to your profile. (This step needs your Kaggle API token in `~/.kaggle/kaggle.json`.)

Generated as a companion to `global-semiconductor-eda.ipynb`. Code excerpts are lightly condensed for readability; the notebook holds the full, runnable source.